Future Generation Computer Systems 176 (2026) 108186



Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs





CyberRAG: An agentic RAG cyber attack classification and reporting tool

Francesco Blefari [a,b,*], Cristian Cosentino [a], Francesco Aurelio Pironti [a], Angelo Furfaro [a], Fabrizio Marozzo [a]

[a] University of Calabria, Via Pietro Bucci, Rende, 87036, Italy
[b] IMT School for Advanced Studies, Piazza San Francesco, Lucca, 55100, Italy

ARTICLE INFO

Keywords:
Large language models
Agentic retrieval-augmented generation
Cyber threat detection
Fine-tuned security classifiers
Intrusion detection systems

ABSTRACT

Intrusion Detection and Prevention Systems (IDS/IPS) in large enterprises can generate hundreds of thousands of alerts per hour, overwhelming analysts with logs requiring rapidly evolving expertise. Conventional machine-learning detectors reduce alert volume but still yield many false positives, while standard Retrieval-Augmented Generation (RAG) pipelines often retrieve irrelevant context and fail to justify predictions. We present CyberRAG, a modular agent-based RAG framework that delivers real-time classification, explanation, and structured reporting for cyber-attacks. A central LLM agent orchestrates: (i) fine-tuned classifiers specialized by attack family; (ii) tool adapters for enrichment and alerting; and (iii) an iterative retrieval-and-reason loop that queries a domain-specific knowledge base until evidence is relevant and self-consistent. Unlike traditional RAG, CyberRAG adopts an agentic design that enables dynamic control flow and adaptive reasoning. This architecture autonomously refines threat labels and natural-language justifications, reducing false positives and enhancing interpretability. It is also extensible: new attack types can be supported by adding classifiers without retraining the core agent. CyberRAG was evaluated on SQL Injection, XSS, and SSTI, achieving over 94 % accuracy per class and a final classification accuracy of 94.92 % through semantic orchestration. Generated explanations reached 0.94 in BERTScore and 4.9/5 in GPT-4-based expert evaluation, with robustness preserved against adversarial and unseen payloads. These results show that agentic, specialist-oriented RAG can combine high detection accuracy with trustworthy, SOC-ready prose, offering a flexible path toward partially automated cyber-defense workflows.

1. Introduction

The cybersecurity landscape has advanced considerably, moving from manual expert-driven processes to increasingly automated and intelligent systems [1]. However, the interpretation and response to cyber threats remains largely semi-automated and dependent on human expertise, particularly in large-scale enterprise environments. Intrusion Detection and Prevention Systems (IDS/IPS) continue to be foundational to network defense [2], but they generate massive volumes of alerts, often hundreds of thousands per hour, many of which require expert validation. Although machine learning-based detectors can reduce this burden, they often suffer from high false positive rates and limited interpretability [3]. Moreover, the output produced by current IDS/IPS is typically presented as raw log strings: highly informative, yet difficult to read and interpret. Logs are static artifacts that lack interactivity and do not provide the possibility for clarification or contextualization. As a result, when an alert is triggered, highly skilled analysts must manually investigate the underlying event, potentially diverting the attention of security teams from truly critical threats. A promising direction to address the aforementioned challenges lies in leveraging LLMs to support the analysis of logs and alerts, enhancing both the readability of results and the analyst's ability to make informed decisions. In recent years, large language models (LLMs) have gained traction in cybersecurity due to their ability to interpret threat data and support analysts through natural language reasoning [4]. A key advancement in this area is RAG [5], which enriches model input with relevant context retrieved from external data sources. This combination improves the quality and grounding of responses [6]. Still, most RAG implementations retrieve context only once before generating output, lacking the ability to refine queries, reason iteratively, or dynamically adapt to complex situations. To push beyond these limitations, autonomous AI agents have emerged as a promising paradigm. These agents are designed to autonomously perform tasks such as continuous monitoring, anomaly detection, and threat mitigation, increasingly becoming a key component of modern security operations [7,8]. When powered by LLMs, such agents can not only interpret security data but also coordinate tools, issue actions, and

* Correspondance author.
E-mail addresses: francesco.blefari@unical.it (F. Blefari), cristian.cosentino@unical.it (C. Cosentino), francesco.pironti@unical.it (F.A. Pironti), angelo.furfaro@unical.it (A. Furfaro), fabrizio.marozzo@unical.it (F. Marozzo).

https://doi.org/10.1016/j.future.2025.108186
Received 25 June 2025; Received in revised form 9 September 2025; Accepted 1 October 2025
Available online 4 October 2025
0167-739X/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

generate reports. However, many LLM-based agents still operate as black boxes, making decisions without exposing their rationale. To overcome the limitations of conventional RAG pipelines and the opacity of LLM-based agents in cybersecurity, we introduce CyberRAG, a modular and extensible agent-based RAG framework for real-time cyber-attack classification, explanation, and reporting. CyberRAG is designed to address two critical requirements: (i) task specialization, through a set of fine-tuned LLM classifiers, each targeting a specific attack category (e.g., DDoS, ransomware, SQL injection); and (ii) context-aware reasoning, enabled by a multi-phase RAG component that iteratively retrieves and refines relevant information from a domain-specific knowledge base. At the center of the framework is a large language model acting as an autonomous agent that orchestrates the classification pipeline, invokes specialized tools as needed, and generates structured, interpretable reports. Unlike standard RAG systems that retrieve context in a single pass, CyberRAG allows the agent to reason over retrieved evidence and requery the knowledge base to refine its understanding. Designed to integrate seamlessly with existing IDS infrastructures, CyberRAG activates upon receiving a flagged alert and autonomously processes the associated network traffic. It identifies the likely attack category, retrieves supporting contextual information, and generates a comprehensive report in natural language that describes the threat and suggests mitigation steps. An integrated LLM-powered assistant enables interactive querying of the report, providing analysts with deeper insights or recommended remediation actions. The system is easily customizable: organizations can expand its internal knowledge base using internal documents, architectural diagrams, or policy files, without the need to fine-tune the agent or classifiers.

The remainder of this paper is organized as follows: Section 2 introduces the topic by presenting foundational cybersecurity concepts, such as web-based attacks, and important LLMs notions including AI agents and Agentic RAG. Section 2.6 discusses the related work. Our methodology is detailed in Section 3, followed by a discussion presenting the Knowledge Bases and the datasets in Section 4. The results are shown in Section 5 and discussed in Section 6. Finally, conclusions are drawn in Section 7.

2. Background

The growing interconnection of web services has made the detection and mitigation of sophisticated cyber-attacks increasingly critical. Organizations today face threats ranging from credential theft to large-scale offensives, such as Distributed Denial of Service (DDoS) [9] attacks and Advanced Persistent Threats (APTs). Among the most critical vulnerabilities, web applications represent a prime target for malicious actors who exploit browser flaws, hijack user sessions, or inject harmful code.

CyberRAG leverages well-known web-based attack vectors to simulate real-world threats and incorporates a collection of established concepts such as LLMs, IDS, AI Agents and Agentic RAG technology which serve as foundational pillars for our approach. These technologies are central to the objectives we aim to achieve and directly influence the design and implementation of our tool. In the following, we provide an overview of the key technologies and methodologies referenced throughout this work.

2.1. Intrusion detection and prevention systems

An *Intrusion Detection System* (IDS) is a crucial component in cybersecurity for identifying ongoing attacks. It functions by monitoring network traffic from the infrastructure it aims to protect, and comparing that traffic against a database of known threats [10]. The two primary detection approaches are signature-based and anomaly-based methods. The *signature-based* approach identifies intrusions by matching network data against predefined patterns, such as specific keywords in HTTP requests or known malicious byte sequences. In contrast, the *anomaly-based* approach leverages machine learning techniques to detect previously unseen threats. These systems learn a model of normal network behavior and raise alerts when deviations from this baseline are observed, potentially indicating malicious activity. When a threat is detected, the IDS generates an alert to notify security operators, who are then responsible for deciding the most appropriate response strategy. Hybrid approaches, aiming at exploiting the advantages of both approaches are known in literature [11]. An *Intrusion Prevention System* (IPS) extends the functionality of an IDS by not only detecting threats but also actively preventing them. Positioned logically, and sometimes physically, between protected hosts and external networks, the IPS intercepts malicious traffic and takes immediate action, such as blocking the connection or isolating affected systems. It uses the same detection techniques as IDS but includes automated response capabilities. A common drawback of IPS solutions is their potential to become a single point of failure in the network, which can impact availability if the system itself is compromised or misconfigured.

2.2. Cyber-attacks

Web-based attacks are among the most dangerous forms of cyber-attacks, as they allow unprivileged users, such as casual visitors to a website, to gain an initial foothold within a corporate network. Once inside, attackers can exploit this access to perform lateral movements across systems, gradually escalating their privileges. By chaining multiple stages of lateral movement and privilege escalation, an attacker can eventually gain full control over the entire system. The ultimate objective of these attacks is either to seize control of the target machine, typically through code injection techniques like SSTI, or to exfiltrate sensitive data, as seen in SQL injection and Cross-Site Scripting (XSS) attacks. These representative attack vectors have been selected to evaluate the capabilities of our tool and are discussed in detail in the remainder of this section.

2.2.1. Cross-Site scripting (XSS)

Cross-Site Scripting (XSS) attacks are a type of security vulnerability that leverages JavaScript code injection [12,13], allowing an external unauthenticated entity to execute the injected code in the victim's web browser. The primary goal of this attack is to steal sensitive information, such as passwords, credit card details, or cookies.

XSS attacks exploit the trust that a web browser places in the content received from a server. Since browsers expect to receive only text, an attacker must find a way to send malicious input to the server, which is then parsed and transmitted back to the browser as part of a web page. When the browser renders this response, it encounters injected HTML tags containing JavaScript code, which it mistakenly executes as legitimate commands. For instance, an attacker might insert malicious code inside `<script></script>` tags to execute arbitrary JavaScript.

There are three main types of XSS attacks:

- *Reflected XSS attacks*, also known as non-persistent XSS, typically involve a specially crafted URL that an attacker tricks a user into visiting. The malicious script is included in the URL and reflected by a vulnerable web page, such as an error page or a search results page, before being executed in the user's browser.
- *Stored XSS attacks*, also known as persistent XSS, occur when a web application stores user-provided input without properly sanitizing it. In these attacks, an attacker injects malicious JavaScript into a website's database (e.g., via a comment field or a user profile). When another user visits the affected page, the script is retrieved from storage and executed in their browser.
- *DOM-based XSS* occurs when an attacker manipulates the Document Object Model (DOM) of a web page using client-side scripts. Unlike other XSS attacks, the malicious payload does not travel through the server; instead, it is executed by modifying the DOM in the user's browser, exploiting vulnerabilities in JavaScript execution.

2.2.2. Server-Side template injection (SSTI)

Server-Side Template Injection (SSTI) is a vulnerability that arises when an attacker is able to inject malicious payloads into a server-side template, exploiting the template engine's capabilities to execute unauthorized code on the server [14]. Many modern web applications utilize template engines to dynamically generate HTML by embedding user inputs into predefined templates. If the application fails to properly sanitize or validate these inputs, the template engine may interpret and execute them as code. This can lead to serious security consequences, including unauthorized data access, exfiltration of sensitive information, or even full remote code execution (RCE) on the server. Commonly used template engines known to be susceptible to SSTI include: Jinja2 (Python), Twig (PHP), Mako (Python), FreeMarker (Java), Velocity (Java). Each of these engines has unique syntax and capabilities, but the underlying vulnerability remains the same: improper handling of untrusted user input. Attackers often start by testing for SSTI through the use of basic expressions. For instance, in the case of Jinja2, an attacker might input the string `{{7*7}}`. If the rendered output from the server contains `49`, it confirms that the expression was evaluated, thus revealing the presence of an exploitable SSTI vulnerability.

2.2.3. SQL injection

A SQL injection is an attack that occurs when an attacker is able to insert and execute malicious code within a vulnerable SQL query, as discussed in [15]. These attacks are commonly used to bypass login mechanisms or to exfiltrate sensitive information from databases.

The primary cause of SQL injection vulnerabilities lies in the use of unsanitized user inputs directly within SQL queries, combined with the lack of secure programming practices such as prepared statements or parameterized queries.

Classical forms of SQL injection include *Piggy-Backed Queries*, where the attacker manipulates input fields by inserting malicious SQL code, often using characters like the semicolon ";" to terminate a legitimate query and append a malicious one. Another typical case is the exploitation of *Stored Procedures*, where attackers create or manipulate database procedures through injected SQL statements. More advanced forms of SQL injection include:

- *Blind SQL Injection*, used when the application does not return visible error messages but allows inference through changes in behavior. This can be further classified into:
  (a) *Boolean-based (content-based)*: the attacker sends boolean conditions in queries and observes the difference in content or structure of the application's response to deduce information.
  (b) *Time-based*: the attacker leverages commands such as `SLEEP()` or `WAITFOR DELAY` to induce time delays, inferring data from the duration of the response.
- *Union-based SQL Injection*, in which the attacker uses the SQL `UNION` operator to combine the result of the original query with another malicious query, effectively retrieving data from other tables.
- *Out-of-Band SQL Injection*, used when traditional techniques fail. This method relies on external systems like DNS or HTTP requests to exfiltrate data. It is particularly effective when the attacker cannot observe the application's response directly but can trigger events on a remote server.

2.3. Large language models

Large Language Models have become one of the most significant advances in Natural Language Processing (NLP), with growing applications in areas such as cybersecurity, medicine, and law [16–20]. These models are typically based on the Transformer architecture [19], which makes it possible to capture long-range dependencies through self-attention mechanisms. Their training usually follows a two-phase paradigm: a large-scale pre-training stage on general-purpose corpora, followed by task-specific fine-tuning on curated datasets. This combination enables LLMs to learn general language representations while remaining adaptable to specialized domains.

From an architectural perspective, LLMs are commonly distinguished into two broad families: encoder-based models and decoder-based models. While they share the same Transformer foundation, their objectives and downstream applications differ substantially. Taken together, the two families highlight complementary strengths: encoder models excel at precise and context-aware classification, while decoder models provide detailed and interpretable explanations.

2.3.1. Encoder-based models

Encoder-based models, exemplified by BERT [17] and RoBERTa [20], are primarily designed for discriminative tasks that require a bidirectional understanding of text. By processing input in both directions, they capture fine-grained semantic relations, which makes them particularly effective for classification, semantic matching, and anomaly detection. In our system, such models are employed as specialized classifiers, each focusing on a distinct type of cyber-attack (e.g., SQL Injection, XSS, SSTI). This specialization allows the classifiers to capture the unique linguistic patterns and syntactic features associated with each attack vector.

2.3.2. Decoder-based models

Decoder-based models, such as GPT [16] and the LLaMA family [21], follow an autoregressive objective, predicting the next token in a sequence. This makes them naturally suited for generative tasks such as text completion, explanation synthesis, and narrative construction. Within our architecture, decoder-based models are integrated into the RAG pipeline, where they generate context-aware descriptions of attacks by combining classifier outputs with relevant external knowledge. While proprietary frontier models such as GPT-4 demonstrate remarkable generative capabilities, they cannot be fine-tuned on domain-specific corpora due to the lack of access to their weights. In highly specialized domains such as cybersecurity, this limits their applicability to zero-shot or few-shot prompting, which generally yields lower and less stable results [22,23]. For this reason, we focus on open-weight encoder models that can be adapted and replicated, while keeping the framework compatible with future open-weight frontier models.

2.4. AI agents

An Agent is a computer system situated in an environment that can autonomously act in its context to reach its delegated objectives [24]. The term *autonomy* refers to the capability and requisites necessary to determine the appropriate course of action to achieve a specified objective. An intelligent agent (AI Agent) is characterized by its ability to perceive its environment, respond to changes within it, initiate actions, and engage with other systems, which may include other agents or human users. One of the core principles foundational to AI agents is the concept of *memory*. Effective memory management improves an agent's capability in maintaining contextual awareness. It enables the agent to draw on previous experiences effectively, thereby facilitating the development of incrementally informed decision-making abilities as time progresses. These concepts are depicted in Fig. 1. As noted in [25], the emergence of LLMs represents another moment of progress in the realization of AI agents. Substantial advances in this direction have culminated in the emergence of LLM agents. In particular, these agents use LLM as reasoning and planning cores to decide the control flow of an application, while maintaining the characteristics of traditional AI agents. LLM agents extend the capabilities of LLMs by facilitating the use of external tools to address specialized tasks, including mathematical computation and code execution. Ultimately, the model evaluates the adequacy of its output and determines whether additional processing is necessary.

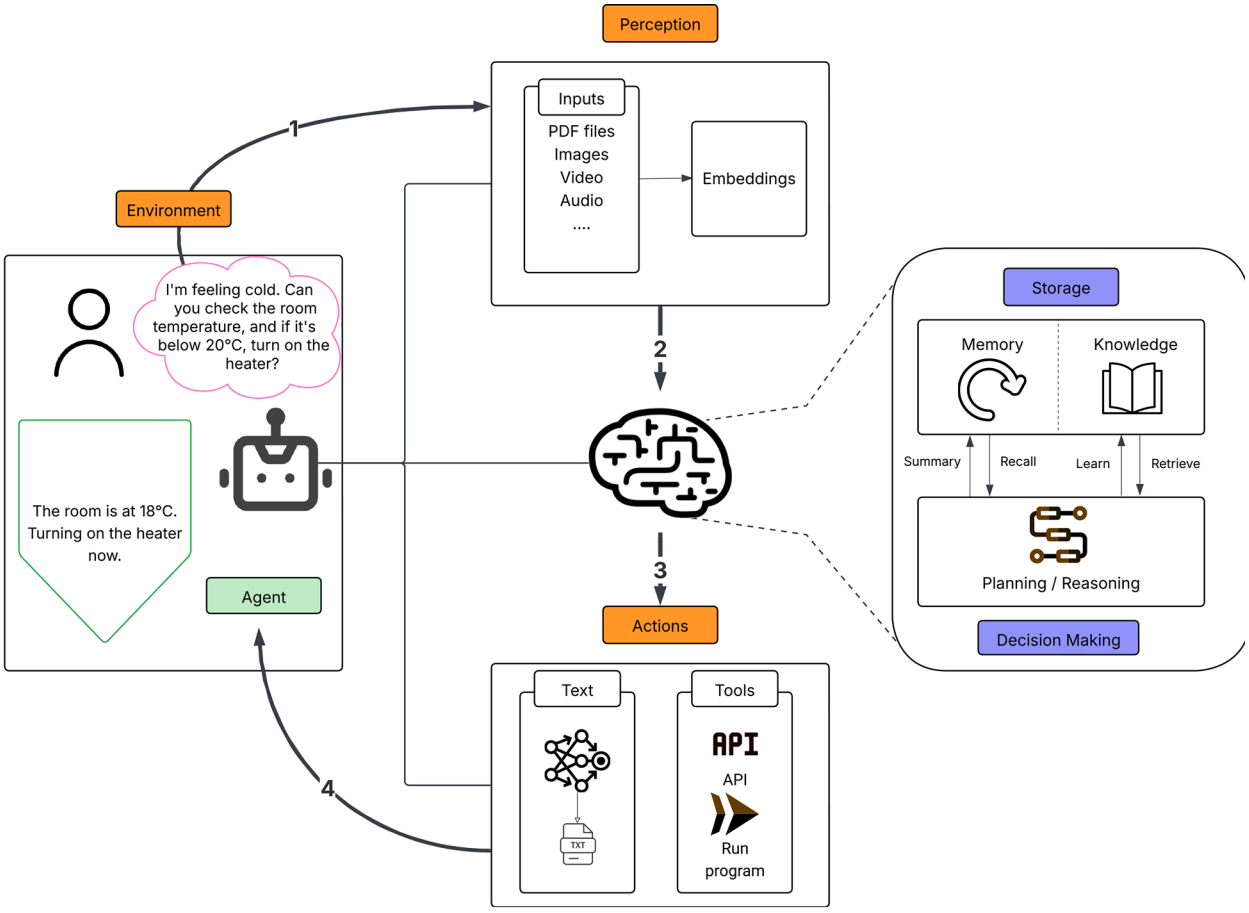


Fig. 1. Structure of a multimodal AI Agent: integrates perception, memory, and planning to interact with the environment and perform intelligent actions.

2.5. Agentic RAG

Although general-purpose LLMs and their associated agents offer remarkable versatility, they frequently lack the *in-depth domain-specific* expertise essential for addressing intricate and specialized problems. One potential solution to this limitation involves retraining or fine-tuning the model; however, undertaking such procedures can be both financially demanding and resource consuming. An alternative strategy to circumvent these challenges emerges from the RAG paradigm. This strategy enhances LLMs by connecting them to external domain-specific knowledge sources, such as document collections, through a retriever component. Functioning like a search engine, the retriever extracts relevant information from this external repository and integrates it directly into the LLM's context. This provides the model with the necessary data to effectively answer complex requests.

In RAG systems, external data is first loaded and segmented into appropriately sized chunks. These chunks are then converted into vector representations (embeddings) and stored in specialized data structures for future retrieval. The typical workflow of a RAG system follows these steps: (i) the user submits a query to the system; (ii) the retriever converts the query into a vector representation and searches for the most relevant stored embeddings, retrieving the corresponding chunks; (iii) the original query is enriched with the retrieved chunks and passed to the LLM; (iv) the LLM generates a context-aware response, which is then presented to the user.

An agent system that utilizes the RAG paradigm is commonly referred to as an Agentic RAG. Several state-of-the-art frameworks, including LangChain [26], LlamaIndex [27], and Langdroid [28], offer user-friendly interfaces for building customized Agentic RAG solutions.

2.6. Attack classification and detection techniques

In Table 1 some key difference between classical and AI-powered techniques are proposed.

2.6.1. Classical techniques

In the context of cyber attack classification and detection systems, "classical" techniques generally focus on the definition of taxonomies, attack patterns, and well-established defensive methods. A traditional aspect involves distinguishing between passive attacks (e.g., eavesdropping and traffic analysis) and active attacks (e.g., injecting malicious packets or tampering with data).

An important example of classification is the AVOIDIT taxonomy, which groups attacks according to vectors, objectives, and impacts, underscoring the need for a holistic perspective on security [29]. Within this framework, threats such as Denial of Service (DoS), port scanning, unauthorized data exfiltration, and, more broadly, data integrity violations are analyzed. In parallel, the literature has also focused on distributed attacks (DDoS), where multiple compromised nodes act simultaneously to deny service or cause damage with high impact [34].

On the prevention and response front, many works focus on the integration of cryptographic tools, authentication systems, and key-management protocols to mitigate the risk of device compromise [35]. This approach often assumes that the primary defensive perimeter is defined by cryptographic robustness and the proper configuration of network services, prioritizing the use of IDS and IPS [30,36].

2.6.2. AI-powered classification and defense techniques

Recent research underscores the growing impact of AI-based tools, particularly Large Language Models , in the domain of cyber-attack or-

Table 1
Comparison between classical and AI-powered Techniques for cyber-attack detection.

Aspect	Classical Techniques	AI-powered Techniques (LLMs)
Detection Approach	Signature-based, rule-driven, taxonomy-defined	Context-aware, data-driven, language understanding
Adaptability to Novel Attacks	Low - limited to known patterns or signatures	High - capable of generalization and zero-day inference
Application Context	Static systems, IDS/IPS, network-layer security	Dynamic classification, forensic analysis, attack generation and simulation
Attack Scope	Focused on DoS, DDoS, passive/active, and layered attacks	Includes phishing, malware, ransomware, social engineering, nation-state threats
Granularity of Analysis	Protocol- and layer-specific, domain-expert encoded	Textual and semantic inference, able to correlate across diverse data types
Key Challenges	Limited scalability, rigid taxonomies, high false positives	Prompt injection risks, adversarial robustness, interpretability
Example Tools / Frameworks	Firewall, IPS/IDS	AutoAttacker, SecBench

Table 2
Summary of related works in cybersecurity detection and classification.

Work	Focus Area	Main Contribution	Remarks
AVOIDIT [29]	Taxonomy of cyber attacks	Proposes a structured classification of attacks by vector, objective, and impact	Foundational for holistic threat modeling
Howard et al. [30]	Intrusion Detection Systems	Explores modern IDS/IPS strategies	Emphasizes signature-based defenses
AutoAttacker [31]	LLM-driven cyber offense	Simulates AI-powered attack chains using GPT-based modules in enterprise settings	Demonstrates offensive use of LLMs
RAG for Cyber Defense [32]	Retrieval-Augmented Generation	Integrates threat intelligence into LLM-based classification via retrieval mechanisms	Improves context relevance and detection accuracy
ChatAPT [33]	Nation-state threat attribution	Uses LLMs with threat intelligence and knowledge graphs for campaign attribution	Supports advanced threat actor profiling

chestration and defense. Early studies demonstrated that LLMs such as ChatGPT could generate attack scripts with success rates ranging from 16 % up to 50 % when combined with basic cybersecurity skills, thereby lowering the barrier to entry for malicious actors [31,37]. Offensive platforms like WormGPT and FraudGPT were explicitly developed to harness LLM capabilities for malicious objectives such as phishing, ransomware development, and malware generation [38].

At the same time, the cybersecurity community has begun to leverage LLMs in a defensive capacity, exploiting their ability to interpret contextual nuances in textual data [37,39]. Advanced frameworks have been proposed to harness LLMs in automated attack classification, attribution, and system hardening. One illustrative example is the *AutoAttacker* system, which uses LLM-guided modules to orchestrate and evaluate attacks on a simulated organizational network [31].

Studies focusing on threats at the *nation-state* level also point to the increasing importance of LLM-driven analysis in attributing sophisticated attacks [33]. Furthermore, reviews of cyber defense LLM applications highlight how these models can enhance intrusion detection, automate forensic analysis, and generate real-time alerts, although persistent challenges remain in interpretability, adversarial robustness, and regulatory compliance [37,38].

A summary of the works discussed above is provided in Table 2.

3. Methodology

The growing sophistication of cyber-attacks necessitates detection systems that are not only accurate but also comprehensible and adaptable. To address these demands, CyberRAG leverages Agentic RAG technology. CyberRAG has a modular architecture whose main component is a general purpose LLM, here referred to as the *core LLM Engine*. This LLM is in charge of handling the overall CyberRAG operation. It directly performs some actions and exploits two other components (tools) for the execution of some specific tasks. The core LLM, by acting as the central intelligent agent that orchestrates the entire system, dynamically manages the two tools and invokes the services offered by other component modules. Each component (tool or module) is responsible for a distinct analytical task, such as classification, contextualization, or user interaction, enabling the Agentic RAG system to dynamically orchestrate reasoning steps in response to uncertainty or ambiguity. This modular design, directed by the core LLM Engine, enhances robustness, scalability, and interpretability.

In contrast to traditional RAG approaches that rely on a single retrieval step, the Agentic RAG framework supports multiple iterative retrieval passes. This allows the system, under the guidance of the Core LLM Engine, to autonomously reassess and refine its initial decisions. For example, if the initial classification is ambiguous or incorrect, CyberRAG can reclassify the input and re-query the appropriate knowledge base, thereby improving both the coherence and accuracy of the final explanation. The CyberRAG's architecture, depicted in Fig. 2, highlights its fundamental components.

The *Classification* tool is in charge of payload classification by employing a set of specific LLM-based classifiers, each fine-tuned on a given attack class (e.g. SQL Injection, XSS, SSTI). Although this work focuses on three classes, CyberRAG is designed for extensibility, allowing the addition of new attack categories with minimal effort.

CyberRAG uses the *RAG tool* to generate a high-quality informative attack description by employing the relevant knowledge associated with the detected attack class. The gathered information is then processed by leveraging the generative capabilities of the core LLM Engine, which transforms technical insights about the detected vulnerability into detailed and contextualized explanations. As a result, CyberRAG not only classifies attacks, but also provides a comprehensive, human-readable narrative report that significantly enhances understanding using the *Attack Description Report Generator* module.

When the core LLM exhibits low confidence in its classification or encounters ambiguity, users can initiate a dialogue with the system (similar to interacting with a chatbot). This enables the system to engage in focused questioning, requesting clarification or additional information before proceeding. This interactive mechanism improves both the accuracy and the interpretability of the system, particularly in complex or edge-case scenarios.

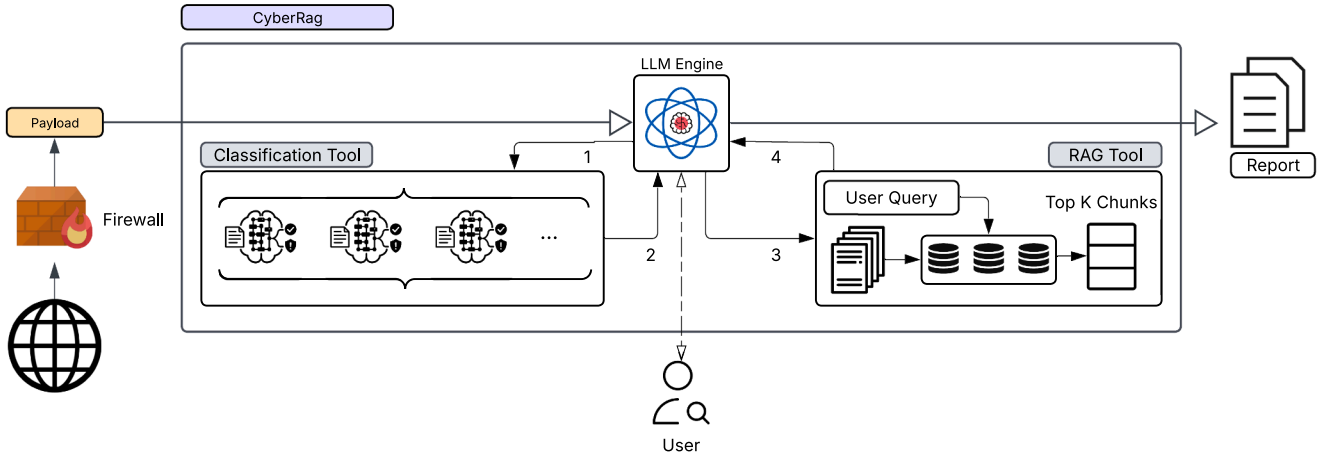


Fig. 2. CyberRAG system architecture: the user interacts with a chatbot connected to the webserver, while the IDS detects attacks from the Internet.

In the following sections, each of these tools and core modules are described in detail. We explain their internal mechanisms, how they interact, and how they contribute to the overall pipeline of cyber-attack interpretation and reporting.

3.1. Classification tool

Once the core LLM engine receives the payload for analysis, it leverages the classification tool to determine the appropriate cyber-attack family to which the payload belongs.

The classification of cyber-attack payloads is managed through a modular ensemble of models based on the BERT family of transformers. Each model in this ensemble is individually fine-tuned to specialize in recognizing a specific type of cyber-attack, enabling targeted detection across a wide range of threat categories.

The fine-tuning process involves adapting a pre-trained language model, initially trained on a large, general-purpose corpus, to a more specific task: identifying particular attack patterns. Through this task-specific training, each model learns to extract and interpret features that are highly relevant to its assigned attack class, enhancing both the precision and robustness of classification, even when payloads exhibit subtle structural variations or deliberate obfuscation.

Within this architecture, each specialized model functions as an independent semantic classifier. Given an input payload, a model produces a classification label indicating the predicted attack type, a confidence score reflecting the certainty of the prediction, and an explanatory component.

The *Classification* tool, to determine the most plausible attack class evaluates the confidence scores produced by each specialized classifier. Rather than applying abstract reconciliation strategies such as majority voting or threshold-based filtering, the LLM-driven decision process directly leverages the highest confidence value as the primary indicator. The class with the highest score is selected as the most reliable prediction and is used to condition the construction of the knowledge-grounded prompt for the retrieval module.

The output of all classifiers is aggregated into a unified structured comparison table. This table enables parallel evaluation of the same payload from multiple semantic perspectives, thus exploiting the complementarity of the specialized models. Such a design ensures scalability and modularity, allowing the addition of other classifiers without disrupting existing components.

The structured output (see Table 3), is then further processed by the core LLM engine as detailed in the following.

3.2. RAG tool

This component enhances the interpretability and informativeness of the system's output by starting from the structured intermediate representation, encapsulating the core features, justifications, and contextual metadata of the attack, produced by the *Classification* tool, and integrating external knowledge sources through the *Retrieval-Augmented Generation* (RAG) mechanism.

The core LLM engine leverages the semantic representation to automatically generate a natural language query that reflects the identified attack type, salient payload characteristics, and relevant contextual indicators. This query is submitted to a semantic search engine that indexes curated cybersecurity resources, including: (i) vulnerability databases (e.g., Common Vulnerabilities and Exposures CVE, National Vulnerability Database NVD), (ii) technical documentation (e.g., Open Worldwide Application Security Project OWASP, MITRE ATT&CK), (iii) scientific literature and incident reports.

To retrieve relevant documents, the system performs a similarity search using dense vector representations stored in three distinct in-memory vector stores, each optimized for a specific source domain. Among the different retrieval strategies, we adopted **Maximal Marginal Relevance (MMR)** due to its ability to balance relevance and diversity [40]. This ensures that the retrieved documents are not only topically relevant to the query but also non-redundant, thereby providing broader contextual coverage.

The top-ranked documents are then summarized, and a narrative contextualization of the attack payload is generated. This includes common usage patterns, associated CVEs, threat severities, and recommended mitigation strategies. The goal is to bridge the gap between

Table 3
Structured output from the *classification tool* (prediction scores per class for each payload).

ID	Payload	SQLi	SSTI	XSS
PD001	`1' and 3580 = ( select count ( * ) from domain.domains as t1, domain.columns as t2, domain.tables as t3 ) --`	0.9999	0.3956	0.0673
PD002	`1''}}{{1016814002+3472965455}} {{'bo'}} {#comment#}{% raw 'bo'.join('7n') %} {{'7n'}}{{3140320242+4078937248}`	0.3999	0.9997	0.3830
PD003	`<time onpointermove=alert(1)>XSS</time>`	0.3998	0.3929	0.9999

low-level payload analysis and high-level cybersecurity knowledge, supporting both automated agents and human analysts.

3.3. Attack description and report generation module

After the payload is classified by the *Classification* tool and relevant information is retrieved by the *RAG* tool, an *Attack Description Report* is generated by leveraging the descriptive capabilities of the core LLM. Aiming to produce a comprehensive narrative of the attack, CyberRAG synthesizes information coming from both the *Classification* tool and the *RAG* tool.

Once the retrieval phase is complete, the core LLM engine composes a structured semantic summary of the incident. This process builds an *attack representation* that captures several key aspects, including:

- the inferred attack type and a justification derived from the model's reasoning;
- salient features of the input payload that contributed to the classification;
- contextual indicators, such as script patterns, DOM elements, or specific input field characteristics that signal malicious behavior;
- high-level knowledge retrieved from external sources, including typical usage scenarios, associated CVEs, threat severity, and recommended mitigation strategies.

This representation is then refined to generate a human-readable attack description. It also provides a solid foundation for downstream tasks such as autonomous decision-making by agent-based systems or integration into reporting and logging pipelines.

By abstracting and reasoning over the outputs of both the classification and retrieval components, this phase significantly enhances system interpretability, supports traceability, and enables explainable decision-making. At the end of this process, a report containing detailed information about the payload is produced.

3.4. User chat

To complement the automated pipeline, the system integrates an interactive *Chat* module that leverages the intrinsic capabilities of the core LLM engine to support natural language exploration of the results. This component allows users to ask questions about classifications, detection mechanisms, and underlying reasoning, thereby enhancing both transparency and interpretability.

The chat interface dynamically adapts its responses based on the user's input and the enriched semantic content generated by the RAG module. It supports: (i) explanations of the classification rationale; (ii) elaborations on attack vectors and behavioral patterns; (iii) response adaptation based on the user's level of expertise, from novice to advanced; (iv) interactive guidance on how to patch the identified vulnerability.

Example: chatbot QA interaction. In the following example, the user engages with the chatbot to gain a clearer understanding of why a specific payload was classified as a Server-Side Template Injection (SSTI) attack.

This interactive dialogue layer enhances system transparency, fostering user trust, comprehension, and overall usability-particularly in high-stakes cybersecurity contexts where interpretability of automated decisions is critical.

Example: SQL Injection Case. The following example illustrates the report generated by CyberRAG after performing the various analysis phases when processing the SQL Injection payload shown below.

Although typically hidden, some LLM engines provide access to their internal reasoning processes. In this example, DeepSeek-R1:7B is used, and its reasoning is presented below.

Chatbot interaction
User: Why was this classified as SSTI?
System: The payload `7*7` is indicative of Server-Side Template Injection (SSTI). It reflects a typical exploit pattern where unescaped user input is evaluated in a template engine, such as Jinja2. Based on semantic similarity and historical pattern recognition, the system assigned a high confidence classification. Supporting information was retrieved from OWASP references and CVE-2020-17526, confirming the vulnerability pattern.

Payload
```
1%"" ) ) waitfor delay '0:0:5' and ( ( ""%"" = ""}
```

Report
Analytical Summary
(1) **String Encapsulation:** The presence of single quotes indicates attempts to break or inject into SQL string boundaries.
(2) **Command-like Patterns:** The use of waitfor delay suggests an attempt to delay response, commonly used in blind SQL injection techniques.
(3) **Syntax Anomalies:** Redundant parentheses and erratic spacing are indicative of non-standard inputs aimed at bypassing filters or altering parser logic.

Conclusion
The payload presents a high-confidence match with known SQL injection signatures. This determination is supported by both syntactic irregularities and functional behaviors that mirror typical exploitation patterns. The classification is further validated through corroborating evidence drawn from authoritative threat intelligence sources.

Feature Vector Summary
• **SQL_keywords_count:** 0
• **Dynamic_values_count:** High
• **SQL_syntax_match:** Low

Reasoning (core LLM)
The system identifies this payload as indicative of a SQL injection attack. The reasoning process highlights multiple suspicious elements: the use of single quotes that typically denote string delimiters in SQL, the presence of a time delay command intended to alter query execution flow, and structurally malformed syntax with excessive and misaligned parentheses. These traits commonly appear in obfuscated or malicious payloads designed to manipulate database queries.

4. Knowledge bases and datasets

4.1. Knowledge bases

For the purposes of this work, we constructed three distinct knowledge bases, one for each type of web-based attack under investigation. The system is designed to distinguish among three specific attack types:

Table 4
Extract of SSTI dataset.

payload	label
{{{ ”._class_._mro_[1]._subclasses_()[59]('/etc/passwd').read() }}}	1
(3a + 1z) + (9b * 9) = 0	0

SSTI, SQL Injection, and Cross-Site Scripting (XSS). To achieve a deep understanding of these attack vectors, dedicated and categorized documents are required.

We sourced our documentation from PortSwigger's Web Security Academy, a well-established educational platform in web security. All relevant materials were downloaded and archived as PDF files. These documents were then organized into folders corresponding to their respective attack categories: `SSTI`, `SQL Injection`, and `XSS`.

Once populated, each knowledge base was processed to enable semantic retrieval. This involved splitting the documents into smaller, manageable chunks. The size of each chunk was set to `800` characters with an overlap of `80` characters to preserve context across boundaries. Each chunk was subsequently transformed into *embeddings*, using a pre-trained Sentence Transformer model, in this instance Sentence-BERT [41].

These embeddings were stored in efficient vector databases, such as in-memory vector stores, using the Faiss library [42], which pairs each chunk with its corresponding embedding. This structure allows for rapid and contextually relevant retrieval based on semantic similarity to user queries or system needs.

4.2. Attacks' dataset

Given the absence of a unified dataset tailored to our specific scenario, we opted to construct a custom dataset by aggregating and adapting existing publicly available resources. Our sources include selected Kaggle datasets [43,44] and the GitHub repository PayloadsAllTheThings, which is widely recognized for its comprehensive collection of real-world attack payloads.

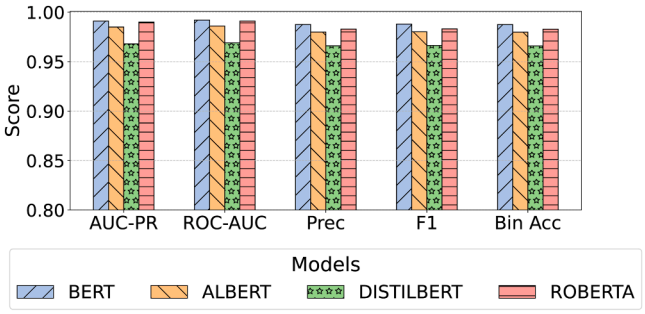
More specifically, for the *SQL Injection* and *Cross-Site Scripting (XSS)* categories, we utilized well-structured datasets available on Kaggle [43,45]. These datasets provide both positive samples (malicious payloads) and negative samples (benign inputs), enabling a supervised learning approach for classification tasks.

For the *Server-Side Template Injection (SSTI)* category, no ready-to-use dataset is available. Consequently, we generated this dataset manually using curated payloads from the PayloadAllTheThings SSTI section in conjunction with outputs from the SSTImap tool. The positive class consists of diverse payloads known to trigger SSTI vulnerabilities across multiple template engines. In contrast, the negative class comprises a variety of benign strings such as mail, names or numbers that do not result in SSTI behavior, serving as clean input examples. This dataset can be downloaded by password at [44] an example is provided in Table 4.

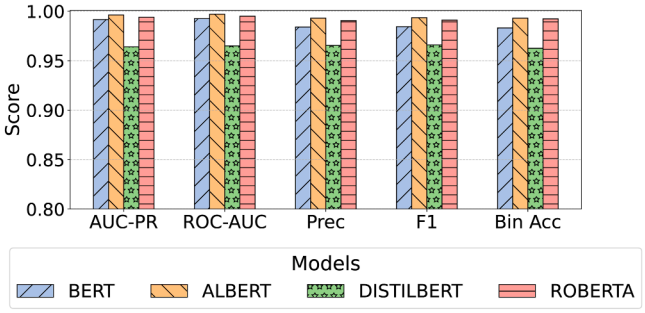
5. Experimental results

This section presents the experimental evaluation of CyberRAG. The goal is to validate: the effectiveness of employing an Agentic RAG approach, the benefits of integrating a retrieval mechanism, and the ability of modern open-weight language models to generate accurate and interpretable threat descriptions.

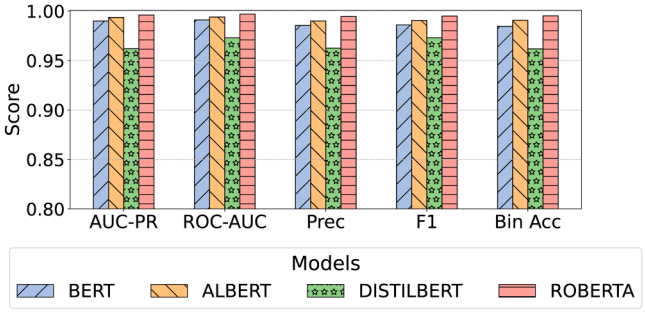
More in-depth, each individual component of CyberRAG has been validated, and an assessment of the performance and reliability of the whole system has been performed. This includes the evaluation of the accuracy of the LLM-based classifiers, the effectiveness of the RAG-based explanation module, and the coherence and usability of the final outputs generated by the complete Agentic RAG pipeline.



(a) SQL Injection.



(b) SSTI.



(c) XSS.

Fig. 3. Classification performance of BERT on different web vulnerabilities using attack-specific training.

5.1. LLM-based classifier for attack identification

To improve the classification of web-based attacks, we designed a system of three independent LLMs, each dedicated to the detection of a specific attack type: SQL Injection, SSTI, and XSS. Rather than relying on a single general-purpose classifier, this architecture allows each model to specialize in learning the nuances, patterns, and syntactic/semantic features of a particular attack category.

Fig. 3 shows the classification performance of the BERT-based classifiers (`bert-base-uncased`, `albert-base-v2`, `distilbert-base-uncased`, `roberta-base`), used to build the classification tool. All models were trained and tested for each attack class considered. Prior to model training, the datasets underwent preprocessing, which involved the removal of incomplete rows, balancing between positive and negative class. Subsequently, each dataset was partitioned into training and test subsets following an 80-20 random split. All classifiers taken into account have been trained with identical hyperparameters: a maximum of 30 epochs and a batch size of 32. The optimizer is RectifiedAdam with a learning rate of 3×10^{-5}. The loss function is binary cross-entropy.

The classifiers were evaluated by using 5 different metrics, three of which are based on the computation of the Area Under the Curve

(AUC) for the precision-recall (PR) and receiving operating characteristic (ROC). The other three are the precision (Prec), the F1-score and the binary accuracy (Bin Acc).

During training, the best weights are saved using early stopping based on validation AUC through a model checkpoint callback.

As observed in Fig. 3(a), the best performing model for the classification of SQL Injection is `bert-base-uncased`. `bert-base-uncased` demonstrates strong generalization on query patterns typical of this attack type. In the case of SSTI, shown in Fig. 3(b), the highest classification performance is achieved by `albert-base-v2`, indicating its greater sensitivity to the subtle templating syntax often involved in such injections. Finally, for XSS detection, depicted in Fig. 3(c), the model that yields the best results is `roberta-base`, which appears to be particularly effective in capturing patterns associated with malicious HTML and JavaScript content.

These observations support the idea that different architectures may offer advantages for distinct types of attack vectors. This specialization strategy is motivated by principles of modular learning, which have shown improved generalization and interpretability in various domains [46,47]. By focusing each classifier on a single task, the system can better capture fine-grained patterns specific to each attack, reducing the risk of overgeneralization.

From a computational efficiency perspective, each classifier was trained on a NVIDIA A100 GPU with identical hyperparameters. On average across the three attack classes, the training time per model was approximately 2.1 h, while dataset preparation (tokenization, balancing, and formatting) required less than 10 min per class. Once trained, inference proved highly efficient: The average prediction and report generation latency was 0.8 s per alert, ensuring the system operates efficiently and meets the time-sensitive demands of enterprise SOC environments.

5.1.1. Model selection rationale and operational constraints

Our evaluation focuses on open-weight encoder models (BERT, RoBERTa, ALBERT, DistilBERT) for two main reasons. First, the artifacts involved in our setting are predominantly textual (payloads, alerts, logs). Encoder architectures remain well suited for fine-grained, bidirectional pattern recognition needed to classify SQLi, XSS, and SSTI, while keeping the training process transparent and reproducible.

Second, open-weight encoders can be fine-tuned on curated security datasets. This capability is essential in a highly specialized and adversarial domain, where domain adaptation materially improves stability and accuracy. By contrast, proprietary frontier models (e.g., GPT-4) do not expose their weights and cannot be fine-tuned; they would need to be used in zero-shot or few-shot configurations only, which typically yields lower and less stable performance for cybersecurity tasks that require targeted adaptation. In addition, closed models entail data egress, cost, and governance constraints that are often incompatible with on-premise, regulated deployments.

5.2. Context-aware Orchestration via RAG

The agentic-RAG receives a structured input consisting of the result of the query to the *RAG* tool, the output probabilities from the three classifiers, and contextual instructions. It is tasked with verifying whether the most probable class is truly correct by checking for known signatures, keywords, and semantic patterns specific to each attack type. In cases of conflicting scores or suspicious payloads that do not align well with any class, the RAG is capable of discarding false positives by evaluating the plausibility of the classification against known patterns. We evaluated this architecture on a curated and unified subset of four popular attack datasets. Each query in the dataset was annotated with a dedicated attribute indicating the reference class label: `SSTI = 1`, `SQL Injection = 2`, and `XSS = 3`, and `0` to denote the absence of an attack. These data were never used during the training phase of either the classifiers or the RAG model.

Generalized Prompt for RAG Decision

System: This model analyzes suspicious queries and identifies the most likely web attack based on classifier outputs and contextual features.

Inputs: Query: `{query}`; Class probabilities: SQL Injection: `{sql_probability}`, SSTI: `{ssti_probability}`, XSS: `{xss_probability}`.

Task: Analyze the query to identify patterns aligned with specific attacks (e.g., SQL keywords, HTML tags, template syntax). Determine the most semantically consistent class, justify the classification based on retrieved knowledge, and produce a class-specific feature vector.

Let $f_{\max}()$ be a function returning the index of the best classifier and c_i the probability returned from the ith classifier

$$f_{\max}(c_1, \dots, c_N) = \begin{cases} \arg \max\limits_{1 \le i \le N} \{c_i\}, & \text{if } \max\limits_{1 \le i \le N} \{c_i\} \ge 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

The initial evaluation using the classifier selected by $f_{\max}(\dots)$ without RAG achieved an accuracy of 84.75 %. However, in some borderline cases, certain models incorrectly classified benign queries as attacks (false positives), likely due to structural similarities with malicious patterns. After integrating the RAG orchestrator, which reasons over context and retrieves relevant evidence, the classification accuracy increases significantly. The best accuracy performance (94.92 %) has been achieved using the `LLaMA3.1:8b` model and the *RAG* tool. The following is an example of a prompt issued by the core LLM to the RAG tool.

5.3. Evaluation of generated explanations

To evaluate the quality of contextualized explanations generated by our RAG framework, we conducted an extensive benchmark across five different open-weight language models, all with approximately 7 billion parameters. The selected models are: **DeepSeek-R1 7B** [48], **Gemma3 4B** [49], **LLaMA3.1 8B** [21], **Mistral 7B** [50], and **Qwen2.5 7B** [51]. All of these models were made available and executed within the `Ollama` environment [52], enabling unified access and deployment for comparative evaluation.[1]

The framework is model-agnostic and forward-compatible: more recent open-weight models (e.g., LLaMA 4, Qwen3) can be integrated as drop-in replacements without structural changes to the system.

For each model, we generated explanations starting from attack payloads and their true class labels. Reference reports were manually curated, describing the attack context, the techniques employed, and key indicators embedded in the payloads. These served as the ground truth for our evaluation.

5.3.1. Evaluation via metrics

The evaluation was designed to assess two core aspects: (i) the fidelity of the generated explanation with respect to the real nature of the attack and its classification, and (ii) the semantic completeness and clarity of the descriptions.

To this end, we employed a suite of well-established metrics from the natural language generation (NLG) literature. Specifically, we used **BLEU** [53], **ROUGE** [54], and **METEOR** [55] for lexical overlap and surface-level comparison. For measuring semantic similarity, we adopted **BERTScore** [56], which leverages contextual embeddings from

[1] At the time the tests were conducted, these represented the state-of-the-art LLMs.

Table 5
Automatic evaluation scores of RAG-generated explanations per model (average across all attack types).

Model	BLEU	ROUGE	METEOR	BERTScore	Factual Consistency
DeepSeek-R1:7B	0.86	0.89	0.84	0.93	0.95
Gemma3:4B	0.84	0.87	0.82	0.92	0.94
LLaMA3.1:8B	0.88	0.90	0.86	0.94	0.96
Mistral:7B	0.85	0.88	0.84	0.93	0.95
Qwen2.5:7B	0.87	0.89	0.85	0.94	0.96

pre-trained language models. Additionally, we introduced a custom factual consistency metric, designed to evaluate the alignment of generated explanations with relevant evidence retrieved by the system.

Table 5 reports the average scores per model, aggregated over explanations for three representative attack types: SQL Injection, Server-Side Template Injection (SSTI), and Cross-Site Scripting (XSS). All models show strong performance in all evaluation metrics, with LLaMA3.1:8B and Qwen2.5:7B slightly outperforming the others in both semantic fidelity and factual alignment.

5.3.2. Evaluation via LLM-as-a-judge

To complement automatic evaluation metrics, we employed a Large Language Model as an external "judge" to provide qualitative ratings of the generated explanations. Inspired by recent evaluation protocols [57,58], this approach uses an independent model to simulate expert-level assessment of textual quality and domain relevance.

Importantly, to avoid bias due to internal feedback loops, we employed **GPT-4** [59] as the external evaluator. This ensures the assessment is detached from the models used for generation and allows for a more objective comparison across outputs.

The evaluation consisted of two complementary components:

(i) **General Explanation Evaluation:** For each generated explanation, GPT-4 rated it on a 1-5 scale for *clarity*, *informativeness*, and *semantic alignment* with the input payload and the attack label.
(ii) **Attack-Specific Evaluation:** GPT-4 assessed each explanation based on how well it captured the characteristics of the corresponding attack type.

The criteria were:

- *Pattern Recognition*: recognition of syntactic or structural cues (e.g., SQL keywords, template markers, script tags).
- *Contextualization*: integration of the payload into a realistic scenario.
- *Terminology Use*: precision and correctness of cybersecurity-related language.

The average scores for each model, aggregated across the three attack types, are shown in Table 6.

The LLM-as-a-Judge results reinforce the trends observed in the automatic evaluations. LLaMA3.1:8B and Qwen2.5:7B received the highest scores in all dimensions, particularly for pattern recognition and precise technical terminology.

5.4. Ablation study: impact of retrieval

To quantify the contribution of the retrieval module to the overall system performance, we conducted an ablation study by comparing the quality of explanations produced with and without the use of RAG. Each explanation was rated on a scale from 1 to 5 by an LLM-judge based on criteria such as semantic completeness, accuracy, and grounding [57,58].

Table 6
GPT-4 based LLM-judge scores (1-5 scale) per model.

Model	Pattern Recognition	Contextualization	Terminology Use	Overall Avg.
DeepSeek-R1 7B	4.9	4.7	4.8	4.8
Gemma3:4B	4.7	4.6	4.7	4.7
LLaMA3.1:8B	5.0	4.8	4.9	4.9
Mistral:7B	4.8	4.7	4.8	4.8
Qwen2.5:7B	4.9	4.8	4.9	4.9

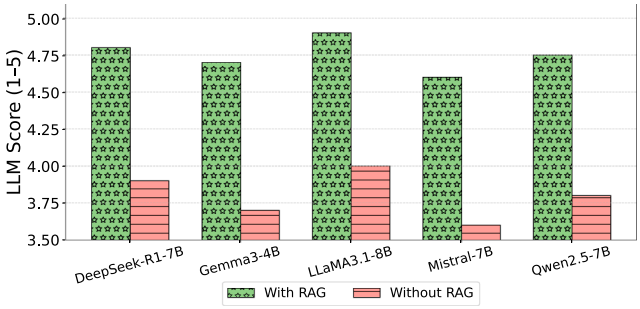


Fig. 4. Comparison between explanations generated with and without retrieval. LLM-based scoring shows consistent advantage from RAG-enhanced generation.

As seen in Fig. 4, explanations produced with RAG consistently score higher across all models, with `LLaMA3.1:8B` reaching the highest mean rating of 4.9. This confirms the added value of retrieval for producing grounded and high-quality justifications.

5.5. Robustness evaluation

To evaluate the robustness of the proposed RAG-based classification system, we designed a controlled experiment targeting two critical scenarios where machine learning models often fail:

(i) **Adversarial examples**, where input queries are subtly perturbed to simulate evasion attempts while preserving their original semantics (e.g., character obfuscation, encoding variations, or token reordering), and (ii) **Out-of-distribution (OOD) inputs**, consisting of queries drawn from attack categories that were not present during training (e.g., Path Traversal, Command Injection), thus requiring the model to reject or correctly classify novel patterns [60,61].

For each language model under evaluation (DeepSeek-R1:7B, Gemma3:4B, LLaMA3.1:8B, Mistral:7B, and Qwen2.5:7B), we constructed two benchmark sets:

- A set of **100 adversarial examples** per attack category, crafted via controlled perturbation techniques.
- A set of **100 OOD queries**, selected from disjoint web attack datasets and annotated to indicate non-membership in the known classes.

The metric used is **Correct Classification (%)**, which quantifies the model's ability to accurately identify or reject inputs under challenging conditions. For each model and scenario, the percentage is computed as:

$$\text{Correct Classification}\,(\%) = \frac{\#\text{Correct Predictions}}{\#\text{Predictions}} \times 100\,\% \tag{1}$$

An adversarial input is considered correctly classified if the model assigns the true attack label despite the perturbation. An OOD query is considered correct if the model abstains from misclassifying it as one of the known attack types.

As shown in Fig. 5, `LLaMA3.1:8B` achieves the highest robustness across both scenarios, correctly classifying 94 % of adversarial examples and 91 % of OOD inputs. `Mistral:7B` also shows strong performance under adversarial conditions (93 %), though with slightly reduced reliability on unseen categories (88.5 %). Other models such as `DeepSeek-R1:7B` and `Gemma3:4B` exhibit a larger performance gap between adversarial and OOD handling, suggesting room for improvement in generalization.

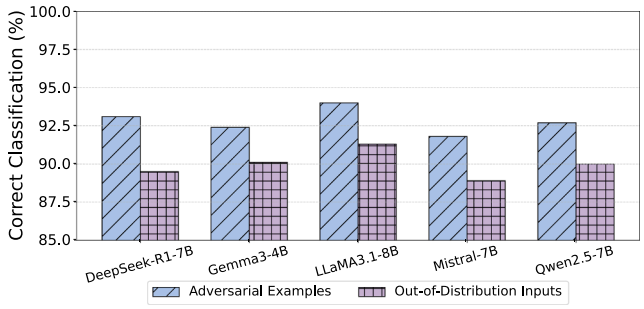


Fig. 5. Evaluation of model robustness based on the percentage of correct classifications under two conditions: (i) *Adversarial Examples*, where inputs are perturbed to simulate evasion attacks, and (ii) *Out-of-Distribution (OOD) Inputs*, representing unseen attack categories. The metric **Correct Classification** (%) reflects the number of accurate predictions out of 100 test cases for each scenario.

6. Discussion

As shown in Section 5 the results obtained with CyberRAG demonstrate the effectiveness of a modular and Agentic RAG approach to the classification and explanation of cyber threats. Delegating the classification task to specialized models, each optimized for a specific attack type, proved more advantageous than a monolithic solution.

The usage of multiple classifiers allows for a modular architecture that contributes to the robustness of the system. If one classifier performs poorly due to data imbalance or ambiguity in one attack category, it does not compromise the performance of the others. This separation of responsibilities leads to a more reliable overall detection pipeline, reducing the likelihood of false positives and false negatives in critical scenarios.

Another advantage is the increased transparency of the classification process. Since each agent operates independently, it is possible to trace back which features or evidence were used to make a decision for each specific attack category. This supports explainability and makes the system more trustworthy in real-world deployments.

In contrast, training a single model to jointly classify all attack types resulted in significantly lower performance, with an overall accuracy of just 0.734. This highlights the limitations of monolithic architectures in dealing with structurally diverse input distributions and further motivates the agent-based decomposition approach.

The general architecture of an Agentic RAG demonstrates the added value of combining probabilistic prediction with contextual and semantic understanding, ensuring a more accurate and explainable decision-making process in critical cybersecurity applications (Section 5.3).

This decoupled architecture enabled better discrimination capability and reduced misclassification, particularly in edge cases. The integration of the RAG component led to a substantial increase in accuracy, from 84.75 % to 94.92 %, highlighting its role as a semantic orchestrator.

A further significant contribution is the RAG component, which improves final decision-making by reasoning over retrieved evidence and domain knowledge. This mechanism not only improves classification accuracy, but also generates technically grounded and context-aware explanations. The synergy between a semantic search engine and a summarization-optimized LLM enables the production of comprehensive reports, as confirmed by both automatic metrics (BLEU, ROUGE, BERTScore) and GPT-4-based evaluation.

The use of a thematic knowledge base, organized by attack type and easily extensible with internal documentation (e.g., PDF files), ensures flexibility and adaptability. In addition, robustness tests under adversarial and out-of-distribution scenarios show resilience, which is critical for real-world deployments. The interactive chat interface further enhances usability by allowing analysts to query the system naturally, receive personalized feedback, and interact with the output according to their level of expertise.

One of the main limitations of this approach is its dependence on the quality of the data within the knowledge base and the precision of the classifier. A poorly maintained knowledge base or an inaccurate classifier can significantly impact the system's effectiveness. In addition, the tool is not equipped to describe new types of attacks or attacks it hasn't been trained to recognize. This limitation makes it less effective against evolving cyber threats, as it can only provide adequate descriptions for attacks that its classifier has been specifically trained on. The effectiveness of CyberRAG is directly dependent on the performance of the preceding IDS/IPS. If the IDS/IPS fails to recognize a security event as potentially dangerous, it won't generate a log entry for that event. Consequently, CyberRAG will have no data to analyze, rendering it useless in that specific scenario. This highlights a critical vulnerability in the system's pipeline, where a failure at the initial detection stage prevents any subsequent analysis. We are aware of this limitation. However, given that CyberRAG is designed to operate in contexts with known threats and its effectiveness is dependent on the Knowledge Base, this issue can be mitigated.

It is sufficient to ensure that the rules for the IDS/IPS are constantly updated. To do this, dedicated online services can be used that offer regularly updated rules and signatures, so as to keep the intrusion prevention and detection system always up to date with the most recent and known threats.

Additional studies on detecting threats not based on a priori known signatures are already present in the literature, but they are considered outside the scope of this work.

Building on the approaches mentioned earlier, recent contributions have introduced specialized RAG-based frameworks tailored specifically for cybersecurity tasks. TechniqueRAG [62] addresses the challenge of adversarial technique annotation in low-resource scenarios by combining off-the-shelf retrievers with instruction-tuned LLMs and a novel zero-shot re-ranking strategy. Its main strength lies in improving precision when labeled data are scarce, while CyberRAG focuses on real-time IDS/IPS alert triage and the generation of structured, SOC-ready reports. AURA [63] introduces a multi-agent architecture for the attribution of APTs, integrating heterogeneous sources such as Trusted Third Parties (TTPs), Indicators of Compromise (IoCs), and malware artifacts. In contrast, CyberRAG targets a different stage of the defense pipeline, aiming to reduce false positives and provide interpretable explanations at the alert triage level. Another complementary direction is CTIKG [64], which enriches RAG pipelines with knowledge graph structures to enhance cyber threat intelligence and contextual reasoning.

7. Conclusion

This work introduces a modular framework for cyber-attack classification and description by integrating a specialized *Classification* tool with a context-aware *RAG* component. CyberRAG merges the precision of fine-tuned large language models, trained to detect specific web-based malicious payload, with the abstraction capabilities and enriched contextual reasoning offered by the RAG pipeline.

Through extensive evaluations, covering both quantitative performance metrics and qualitative interpretability assessments, CyberRAG consistently outperforms traditional monolithic classifiers in terms of accuracy, robustness, and explainability. The framework is particularly effective in handling complex, noisy, or ambiguous inputs, and it demonstrates strong generalization capabilities across previously unseen payloads, highlighting its adaptability and reliability in dynamic cybersecurity contexts.

CyberRAG represents a step forward in intelligent automation for incident response, providing security analysts with a reliable, adaptable, and semantically enriched assistant capable of acting as a virtual cybersecurity expert. Its modular architecture supports seamless integration with existing security infrastructures, enabling organizations

to enhance their detection and response workflows without major architectural changes. Furthermore, this modularity lays a solid foundation for future extensions, such as incorporating new analytical components, integrating with external threat intelligence sources, or adapting to evolving attack surfaces and organizational needs.

Looking ahead, future developments will aim to expand the scope of supported attack types by expanding the underlying taxonomy and refining detection capabilities. A key area of enhancement involves the integration of structured knowledge sources, such as knowledge graphs, to support more advanced, explainable, and context-aware reasoning. In parallel, efforts will be directed toward enabling controlled and auditable automated response mechanisms, allowing CyberRAG to act not only as an analytical assistant but also as an intelligent orchestrator of defensive actions. These advancements will encourage greater integration with Security Information and Event Management (SIEM) platforms and threat intelligence pipelines, ultimately leading to a more autonomous, scalable, and proactive cybersecurity defense architecture. Overall, CyberRAG emerges as a robust, innovative, and forward-looking prototype, that sets the foundation for the next generation of intelligent cybersecurity tools, combining analytical rigor with actionable insight.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used ChatGPT in order to check for grammar errors, typos, and overall writing clarity. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

CRediT authorship contribution statement

Francesco Blefari: Writing – review & editing, Writing – original draft, Validation, Methodology, Conceptualization; **Cristian Cosentino:** Writing – original draft, Visualization, Investigation, Data curation; **Francesco Aurelio Pironti:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Conceptualization; **Angelo Furfaro:** Writing – review & editing, Supervision, Methodology, Funding acquisition, Formal analysis; **Fabrizio Marozzo:** Visualization, Validation, Supervision, Data curation.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests Angelo Furfaro reports financial support was provided by projects SERICS (PE00000014) and FAIR (PE00000013). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was partially supported by the projects SERICS (PE00000014) and FAIR (PE0000013) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU. The work of Francesco A. Pironti was supported by Agenzia per la cybersicurezza nazionale under the 2024–2025 funding programme for promotion of XL cycle PhD research in cybersecurity (CUP H23C24000640005).

References

[1] N. Mohamed, Artificial intelligence and machine learning in cybersecurity: a deep dive into state-of-the-art techniques and future paradigms, Knowl. Inf. Syst. 67 (2025) 6969–7055. https://doi.org/10.1007/s10115-025-02429-y
[2] H.-J. Liao, C.-H. Richard Lin, Y.-C. Lin, K.-Y. Tung, Intrusion detection system: a comprehensive review, J. Netw. Comput. Appl. 36 (1) (2013) 16–24. https://doi.org/10.1016/j.jnca.2012.09.004
[3] F. Blefari, F.A. Pironti, A. Furfaro, Toward a log-based anomaly detection system for cyber range platforms, in: The 19Th International Conference on Availability, Reliability and Security (ARES 2024), ACM, 2024. https://doi.org/10.1145/3664476.3669976
[4] F.N. Motlagh, M. Hajizadeh, M. Majd, P. Najafi, F. Cheng, C. Meinel, Large language models in cybersecurity: state-of-the-art, 2024, arXiv:2402.00891
[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS'20, Curran Associates Inc., Red Hook, NY, USA, 2020.
[6] M. Mudassar Yamin, E. Hashmi, M. Ullah, B. Katt, Applications of LLMs for generating cyber security exercise scenarios, IEEE Access 12 (2024) 143806–143822. https://doi.org/10.1109/ACCESS.2024.3468914
[7] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, Y. Xiang, AI Agents under threat: a survey of key security challenges and future pathways, ACM Comput. Surv. 57 (7) (2025). https://doi.org/10.1145/3716628
[8] A. Behera, A. Vedashree, M.R. Kumar, K. Upreti, Artificial intelligence and cybersecurity prospects and confronts, in: Navigating Cyber-Physical Systems with Cutting-Edge Technologies, IGI Global Scientific Publishing, 2025, pp. 155–184. https://doi.org/10.4018/979-8-3693-5728-6.ch006
[9] A. Furfaro, P. Pace, A. Parise, Facing DDos bandwidth flooding attacks, Simul. Modell. Pract. Theory 98 (2020) 101984. https://doi.org/10.1016/j.simpat.2019.101984
[10] H. Debar, An introduction to intrusion-detection systems in: Proceedings of Connect, (2000), pp. 1–18.
[11] F. Angiulli, L. Argento, A. Furfaro, A. Parise, A hierarchical hybrid framework for modelling anomalous behaviours, Simul. Modell. Pract. Theory 82 (2018) 103–115. https://doi.org/10.1016/j.simpat.2017.12.013
[12] S. Gupta, B.B. Gupta, Cross-site scripting (XSS) attacks and defense mechanisms: classification and state-of-the-Art, Int. J. Syst. Assur. Eng. Manage. 8 (1) (2017) 512–530. https://doi.org/10.1007/s13198-015-0376-0
[13] I. Hydara, A.B.M. Sultan, H. Zulzalil, N. Admodisastro, Current state of research on cross-site scripting (XSS) – A systematic literature review, Inf. Softw. Technol. 58 (2015) 170–186. https://doi.org/10.1016/j.infsof.2014.07.010
[14] Y. Zhao, Y. Zhang, M. Yang, Remote code execution from SSTI in the sandbox: automatically detecting and exploiting template escape bugs, in: 32Nd USENIX Security Symposium (USENIX Security 23), USENIX Association, Anaheim, CA, 2023, pp. 3691–3708. https://www.usenix.org/conference/usenixsecurity23/presentation/zhao-yudi.
[15] J.P. Singh, Analysis of SQL injection detection techniques, Theor. Appl. Inf. 28 (2016). https://doi.org/10.20904/281-2037
[16] T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D.M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: Proceedings of the 34Th International Conference on Neural Information Processing Systems, NIPS'20, Curran Associates Inc., Red Hook, NY, USA, 2020.
[17] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, in: J. Burstein, C. Doran, T. Solorio (Eds.), Proc. of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), Association for Computational Linguistics, Minneapolis, Minnesota, 2019, pp. 4171–4186. https://doi.org/10.18653/v1/N19-1423
[18] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, Language models are unsupervised multitask learners, OpenAI Blog 1 (8) (2019) 9. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31St International Conference on Neural Information Processing Systems, NIPS'17, Curran Associates Inc., Red Hook, NY, USA, 2017, p. 6000–6010.
[20] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, RoBERTa: a robustly optimized BERT pretraining approach (2019). arXiv:1907.11692
[21] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Roziére, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: open and efficient foundation language models, 2023, arXiv:2302.13971
[22] C. Cosentino, M. Gündüz-Cüre, F. Marozzo, Ş. Öztürk-Birim, Exploiting large language models for enhanced review classification explanations through interpretable and multidimensional analysis, in: International Conference on Discovery Science, Springer, 2024, pp. 3–18.
[23] R. Cantini, C. Cosentino, F. Marozzo, Multi-dimensional classification on social media data for detailed reporting with large language models, in: IFIP International Conference on Artificial Intelligence Applications and Innovations, Springer, 2024, pp. 100–114.
[24] M. Wooldridge, An Introduction to MultiAgent Systems, Wiley, 2nd edition, 2009.

[25] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Qin, Y. Zheng, X. Qiu, X. Huang, Q. Zhang, T. Gui, The rise and potential of large language model based agents: a survey, Sci. China Inf. Sci. 68 (2025). https://doi.org/10.1007/s11432-024-4222-0
[26] H. Chase, LangChain, 2022, https://github.com/langchain-ai/langchain.
[27] J. Liu, LlamaIndex, 2022, https://github.com/jerryjliu/llama_index.
[28] P. Chalasani, S. Jha, Langdroid, 2025 https://github.com/langroid/langroid.
[29] C. Simmons, C. Ellis, S. Shiva, D. Dasgupta, Q. Wu, AVOIDIT: A Cyber Attack Taxonomy, Technical Report Supported under grant N00014-09-1-0752, Office of Naval Research (ONR), 2009.
[30] F. Howard, Modern web attacks, Netw. Secur. 2008 (4) (2008) 13–15. https://doi.org/10.1016/S1353-4858(08)70053-9
[31] J. Xu, J.W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, Z. Li, AutoAttacker: a large language model guided system to implement automatic cyber-attacks, 2024, arXiv:2403.01038
[32] S. Rajapaksha, R. Rani, E. Karafili, A RAG-Based question-answering solution for cyber-attack investigation and attribution, in: Computer Security. ESORICS 2024 International Workshops, Springer Nature Switzerland, Cham, 2025, pp. 238–256.
[33] K. Guru, ChatAPT: applying large language models (LLMs) for nation-state cyber attack attribution, Stanford Digital Repository (2024). https://purl.stanford.edu/tk104mm3260.
[34] A. Singh, B.B. Gupta, Distributed denial-of-service (DDos) attacks and defense mechanisms in various web-enabled computing platforms: issues, challenges, and future research directions, Int. J. Semant. Web Inf. Syst. 18 (1) (2022) 1–43. https://doi.org/10.4018/IJSWIS.297143
[35] B.W. Cashell, W.D. Jackson, M. Jickling, B. Webel, The Economic Impact of Cyber-Attacks, Technical Report, CRS Report for Congress, 2004. https://digital.library.unt.edu/ark:/67531/metadc817913/.
[36] B. Jovicic, D. Simic, Common web application attack types and security using ASP.NET, Comput. Sci. Inf. Syst. 3 (2) (2006) 83–96. https://doi.org/10.2298/CSIS0602083J
[37] M.A. Ferrag, F. Alwahedi, A. Battah, B. Cherif, A. Mechri, N. Tihanyi, T. Bisztray, M. Debbah, Generative AI in cybersecurity: a comprehensive review of LLM applications and vulnerabilities, Internet Things Cyber-Phys. Syst. 5 (2025) 1–46. https://doi.org/10.1016/j.iotcps.2025.01.001
[38] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, D. Meng, When LLMs meet cybersecurity: a systematic literature review, Cybersecurity 8 (1) (2025) 55. https://doi.org/10.1186/s42400-025-00361-w
[39] Y. Chen, M. Cui, D. Wang, Y. Cao, P. Yang, B. Jiang, Z. Lu, B. Liu, A survey of large language models for cyber threat detection, Comput. Secur. 145 (2024) 104016. https://doi.org/10.1016/j.cose.2024.104016
[40] J. Carbonell, J. Goldstein, The use of MMR, diversity-based reranking for reordering documents and producing summaries, in: Proceedings of the 21St Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 1998, pp. 335–336. https://doi.org/10.1145/290941.291025
[41] N. Reimers, I. Gurevych, Sentence-BERT: sentence embeddings using siamese BERT-Networks, in: K. Inui, J. Jiang, V. Ng, X. Wan (Eds.), Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9Th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Association for Computational Linguistics, Hong Kong, China, 2019, pp. 3982–3992. https://doi.org/10.18653/v1/D19-1410
[42] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, H. Jégou, The Faiss library, 2025, arXiv:2401.08281
[43] Kaggle, XSS dataset, 2025, (https://www.kaggle.com/datasets/syedsaqlainhussain/cross-site-scripting-xss-dataset-for-deep-learning).
[44] F. Blefari, F.A. Pironti, SSTI dataset, 2025, (https://github.com/francescopirox/ssti_dataset).
[45] Kaggle, Sql injection dataset, 2025, (https://www.kaggle.com/datasets/syedsaqlainhussain/sql-injection-dataset/data).
[46] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean, Outrageously large neural networks: the sparsely-gated mixture-of-experts layer, 2017, arXiv:1701.06538
[47] J. Andreas, D. Klein, S. Levine, Modular multitask reinforcement learning with policy sketches, in: Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML'17, JMLR.org, 2017, p. 166–175.
[48] DeepSeek-AI, DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning, 2025, arXiv:2501.12948
[49] Google DeepMind, Gemma: open models based on gemini research and technology, accessed: 2024-03, 2024, https://ai.google.dev/gemma.
[50] A.Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D.S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L.R. Lavaud, M.-A. Lachaux, P. Stock, T.L. Scao, T. Lavril, T. Wang, T. Lacroix, W.E. Sayed, Mistral 7B, 2023, arXiv:2310.06825
[51] A. Cloud, Qwen2: a family of open-source language models by Alibaba Cloud, 2024, https://huggingface.co/Qwen/Qwen2-7B.
[52] O. Project, Ollama: run LLMs locally, 2024, https://ollama.com.
[53] K. Papineni, S. Roukos, T. Ward, W.-J. Zhu, BLEU: A method for automatic evaluation of machine translation, in: Proceedings of the 40Th Annual Meeting on Association for Computational Linguistics, ACL '02, Association for Computational Linguistics, USA, 2002, p. 311–318. https://doi.org/10.3115/1073083.1073135
[54] C.-Y. Lin, ROUGE: A package for automatic evaluation of summaries, in: Proc. of Workshop on Text Summarization Branches Out, 2004, pp. 74–81. https://aclanthology.org/W04-1013/.
[55] A. Lavie, A. Agarwal, Meteor: an automatic metric for MT evaluation with high levels of correlation with human judgments, in: Proc. of the Second Workshop on Statistical Machine Translation, StatMT '07, Association for Computational Linguistics, USA, 2007, p. 228–231. https://aclanthology.org/W07-0734/.
[56] T. Zhang, V. Kishore, F. Wu, K.Q. Weinberger, Y. Artzi, BERTScore: Evaluating text generation with BERT, in: International Conference on Learning Representations (ICLR), 2020.
[57] T. Goyal, J.J. Li, G. Durrett, News summarization and evaluation in the era of GPT-3, 2023, arXiv:2209.12356
[58] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, C. Zhu, G-Eval: NLG evaluation using GPT-4 with better human alignment, in: H. Bouamor, J. Pino, K. Bali (Eds.), Proc. of the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Singapore, 2023, pp. 2511–2522. https://doi.org/10.18653/v1/2023.emnlp-main.153
[59] OpenAI, GPT-4 technical report, 2023, (https://openai.com/research/gpt-4).
[60] M.T. Ribeiro, T. Wu, C. Guestrin, S. Singh, Beyond accuracy: behavioral testing of NLP models with checklist (2020) 4902–4912. https://doi.org/10.18653/v1/2020.acl-main.442
[61] D. Hendrycks, X. Liu, E. Wallace, A. Dziedzic, R. Krishnan, D. Song, Pretrained transformers improve out-of-distribution robustness, in: D. Jurafsky, J. Chai, N. Schluter, J. Tetreault (Eds.), Proceedings of the 58Th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Online, 2020, pp. 2744–2751. https://doi.org/10.18653/v1/2020.acl-main.244
[62] A. Lekssays, U. Shukla, H.T. Sencar, M.R. Parvez, TechniqueRAG: retrieval augmented generation for adversarial technique annotation in cyber threat intelligence text, in: W. Che, J. Nabende, E. Shutova, M.T. Pilehvar (Eds.), Findings of the Association for Computational Linguistics: ACL 2025, Association for Computational Linguistics, Vienna, Austria, 2025, pp. 20913–20926. https://doi.org/10.18653/v1/2025.findings-acl.1076
[63] N. Rani, S.K. Shukla, AURA: a multi-agent intelligence framework for knowledge-enhanced cyber threat attribution, 2025, arXiv:2506.10175
[64] L. Huang, X. Xiao, CTIKG: LLM-powered knowledge graph construction from cyber threat intelligence, in: First Conference on Language Modeling, 2024. https://openreview.net/forum?id=DOMP5AgwQz.